

Dokumentacja końcowa

- Mikołaj Garbowski
- Mariusz Pakulski
- Paweł Łasica

Definicja problemu

Badane zagadnienie to agent oparty na wielkim modelu językowym przeznaczony do konwersacyjnej analizy danych ustrukturyzowanych. Kluczowym komponentem takiego systemu jest zadanie text-to-SQL, czyli tłumaczenie zapytania użytkownika w języku naturalnym na zapytanie SQL. Agent ma możliwość wykonywania zapytań SQL w samej bazie danych, a także mechanizmy dostępu do informacji na temat analizowanej bazy danych (np. schemat, przykładowe dane).

W ramach projektu porównujemy skuteczność różnych strategii agenta oraz różnych LLM na benchmarku Spider [1]

Studia literaturowe

Zagadnienie wykorzystania modeli językowych do zastosowań z zakresu analizy danych i zadań BI dla baz relacyjnych jest szeroko badane. Typowe rozwiązania obejmują podejścia typu text-to-sql, a także text-to-python i text-to-dsl jako popularne alternatywy do obrabiania ustrukturyzowanych danych [2].

W celu zrozumienia struktury tabeli, typowo dane tabelaryczne są serializowane do jakiejś postaci tekstowej (markdown, json, itp.) co pozwala na bezpośrednie umieszczenie danych w prompcie, natomiast wyzwaniem jest ograniczony rozmiar kontekstu. Badane są także alternatywne podejścia, np. przetwarzanie obrazu tabeli modelami typu VLM, przetwarzanie tabeli w postaci struktur grafowych, a także szczególnie interesujące: traktowanie danych tablicywnych jako odrębnej modalności. Przykładem takiego rozwiązania jest TableGPT2 [3], które łączy koder danych tabelarycznych, produkujący semantyczne osadzenia dla kolumn tabeli, połączony z modelem językowym typu tylko-dekoder wykorzystującym te osadzenia oraz prompt użytkownika.

Sposobem na rozwiązanie problemu z rozmiarem kontekstu, wzbogacenie promptu użytkownika o wiedzę domenową, a także wydobywanie informacji o schemacie bazy danych, stosowane są metody znane z zastosowań typu RAG, zadania *retrieval*.

Zbiory danych

Popularny zbiór WikiSQL [4] obejmuje proste zapytania SQL dotyczące pojedynczych tabel.

Bardziej złożone zapytania SQL (obejmujące złączenia, agregacje itp.) zawierają, znacznie większe zbiory, Spider [1] i bardziej obszerny BIRD [5]. rny Spider [6].

Miary jakości

W literaturze stosowane są następujące miary jakości do oceny rozwiązań typu text-to-SQL:

- Execution accuracy [6] - Określa zgodność wyników zapytania wygenerowanego przez model z wynikami zapytania referencyjnego. Zapytanie uznaje się za poprawne, jeśli po wykonaniu na bazie danych zwraca taki sam rezultat jak zapytanie wzorcowe.
- Exact set match accuracy [6] - Polega na porównaniu struktury zapytania SQL wygenerowanego przez model z zapytaniem referencyjnym. Zapytanie uznaje się za poprawne, jeżeli wszystkie jego komponenty, takie jak SELECT, WHERE, GROUP BY czy ORDER BY, odpowiadają komponentom zapytania wzorcowego.
- Execution Accuracy per SQL Hardness [1] - Dzieli zapytania SQL według poziomu złożoności na cztery kategorie:
 - easy,
 - medium,
 - hard,
 - extra hard.

Klasyfikacja ta opiera się na analizie struktury zapytania SQL, w szczególności na obecności:

- złączeń tabel (JOIN),
 - grupowania danych (GROUP BY),
 - zapytań zagnieżdżonych,
 - operacji zbiorowych,
 - liczby warunków i agregacji.
- Component matching [1], [7] - Umożliwia analizę poprawności poszczególnych komponentów zapytania SQL takich jak:
 - SELECT,
 - WHERE,
 - GROUP BY,
 - ORDER BY,
 - JOIN,
 - nested SQL.

Strategie agentów

Istnieje wiele strategii działania agenta, które różnią się sposobem generowania zapytań SQL, interakcją z bazą danych oraz mechanizmami weryfikacji i poprawy błędów. Niektóre z nich to:

- Plan and Solve [8] - polega na rozdzieleniu procesu rozwiązywania problemu na dwa etapy - utworzenie planu rozwiązania i wykonanie zaplanowanych

kroków. Model językowy najpierw generuje strukturę działania opisującą sposób rozwiązania zadania, a następnie realizuje kolejne kroki prowadzące do wygenerowania zapytania SQL.

- Chain-of-thought [9] - klasyczna wersja polega na generowaniu rozwiązania poprzez sekwencję kroków rozumowania, bez bezpośredniej interakcji z narzędziami lub środowiskiem wykonawczym w trakcie procesu wnioskowania. Model językowy analizuje zapytanie użytkownika i generuje końcowe zapytanie SQL w jednym przebiegu.
- ReAct - Strategia ReAct (Reason and Act) [9] łączy proces rozumowania z wykonywaniem akcji w środowisku zewnętrznym. Model językowy generuje kolejne kroki działania, wykonuje zapytania SQL lub operacje pomocnicze, analizuje wyniki ich wykonania oraz w razie potrzeby modyfikuje swoje decyzje. W projekcie zaimplementowano uproszczony wariant tej idei, nazwany ReAct-lite, w którym akcja została ograniczona do wygenerowania jednego zapytania SQL, a obserwacją jest wynik wykonania zapytania lub komunikat błędu.

Self-correction zapytań SQL

W projekcie self-correction jest uruchamiany dopiero wtedy, gdy wygenerowane zapytanie SQL nie wykona się poprawnie albo zostanie odrzucone przez walidator. Model otrzymuje wtedy pytanie użytkownika, schemat bazy, poprzednie zapytanie oraz komunikat błędu i ma wygenerować poprawioną wersję SQL. Mechanizm ten jest stosowany zero-shot, bez dodatkowego trenowania modelu na danych specyficznych dla zadania.

Opis rozwiązania

Projekt obejmuje implementację agentów wykorzystujących różne strategie oraz porównanie ich skuteczności.

Zbiór danych

Do ewaluacji jakości wykorzystujemy zbiór Spider. WikiSQL uznajemy za nieodpowiedni dla naszego projektu, skoncentrowanego na bardziej złożonych zapytaniach analitycznych. BIRD byłby również dobrym wyborem, natomiast decydujemy się na Spider: również bardzo popularny i mniejszy, co ułatwi nam pracę na ograniczonych zasobach sprzętowych.

Baza danych

Integrujemy agenta z SQLite: uruchamianą lokalnie, relacyjną bazą danych. Jest to typowy wybór przy tego typu zadaniach, używana w popularnych zbiorach danych [1], [4], [5].

Plan eksperymentów

Porównujemy skuteczność trzech strategii generowania zapytań SQL:

- Chain-of-Thought,
- Plan-and-Solve,
- ReAct-lite, czyli uproszczonego wariantu ReAct z iteracyjną korektą zapytań SQL.

Dla każdej strategii analizujemy execution accuracy, exact match accuracy oraz metryki partial matching dla komponentów SQL. Główne wyniki liczymy na zbiorze testowym Spider obejmującym 2147 przykładów. Dodatkowo używamy próbki `diagnostic_test_40`, zrównoważonej po poziomach trudności, do szybkiego testowania zmian w promptach i postprocessingu.

Podstawowe eksperymenty wykonujemy na dwóch małych modelach uruchamianych lokalnie:

- Qwen/Qwen2.5-1.5B-Instruct,
- meta-llama/Llama-3.2-1B-Instruct.

Dodatkowo testujemy większy model `qwen/qwen3-32b` przez API Groq, aby sprawdzić wpływ jakości modelu na te same strategie. Wariant ReAct-lite uruchamiamy z dwiema próbami korekty, co stanowi kompromis między self-correction a czasem inferencji.

Implementacja

Platforma

Modele lokalne uruchamiamy z wykorzystaniem biblioteki `transformers` oraz adapterów HuggingFace. Daje to kontrolę nad modelem, tokenizerem i parametrami generowania, ale ogranicza wybór modeli do takich, które mieszczą się w pamięci GPU.

Dodatkowo dodano adapter do API Groq dla modelu `qwen/qwen3-32b`. Prompt nadal jest definiowany w naszym kodzie, natomiast Groq służy wyłącznie jako zewnętrzna warstwa inferencji. Dla tego modelu zastosowano format SQL-only, aby uniknąć zapisywania w predykcjach komentarzy lub znaczników reasoningowych. Przetestowano `qwen/qwen3-32b` dla wszystkich opisanych wariantów rozumowania.

Narzędzia

- langchain/langgraph [10], [11] - biblioteki dostarczające rozwiązania dotyczące budowania promptów, agentów oraz zdolności rozumowania
- transformers - biblioteka ułatwiająca pracę z wielkimi modelami językowymi
- pytorch - biblioteka ogólnego przeznaczenia do działań głębokiego uczenia oraz tensorów

Przepływ działania agenta

Ogólny schemat działania agenta jest przedstawiony poniżej. Konkretnie warianty wprowadzają pewne modyfikacje do tego schematu, ale ogólna struktura pozostaje zbliżona.

Krok	Komponent	Rola
1	<code>list_tables</code>	Pobranie listy tabel dostępnych w bazie danych.
2	<code>select_tables</code>	Wybór tabel istotnych dla pytania użytkownika.
3	<code>get_schema</code>	Pobranie schematu wybranych tabel.
4	<code>generate_query</code>	Wygenerowanie zapytania SQLite <code>SELECT</code> .
5	<code>execute_query</code>	Sprawdzenie bezpieczeństwa i wykonanie zapytania.
6	<code>use_all_tables</code>	Awaryjne użycie pełnego schematu, jeżeli wcześniejszy wybór tabel był niewystarczający.
7	<code>correct_query</code>	Korekta zapytania na podstawie błędu.
8	<code>generate_answer</code>	Wygenerowanie krótkiej odpowiedzi dla użytkownika.

W trybie ewaluacyjnym agent może kończyć działanie po wygenerowaniu i ewentualnej korekcie SQL. Jest możliwość wygenerowania odpowiedzi dla użytkownika w języku naturalnym, ale nie podlega to ocenie w benchmarku.

Wariant Chain-of-Thought

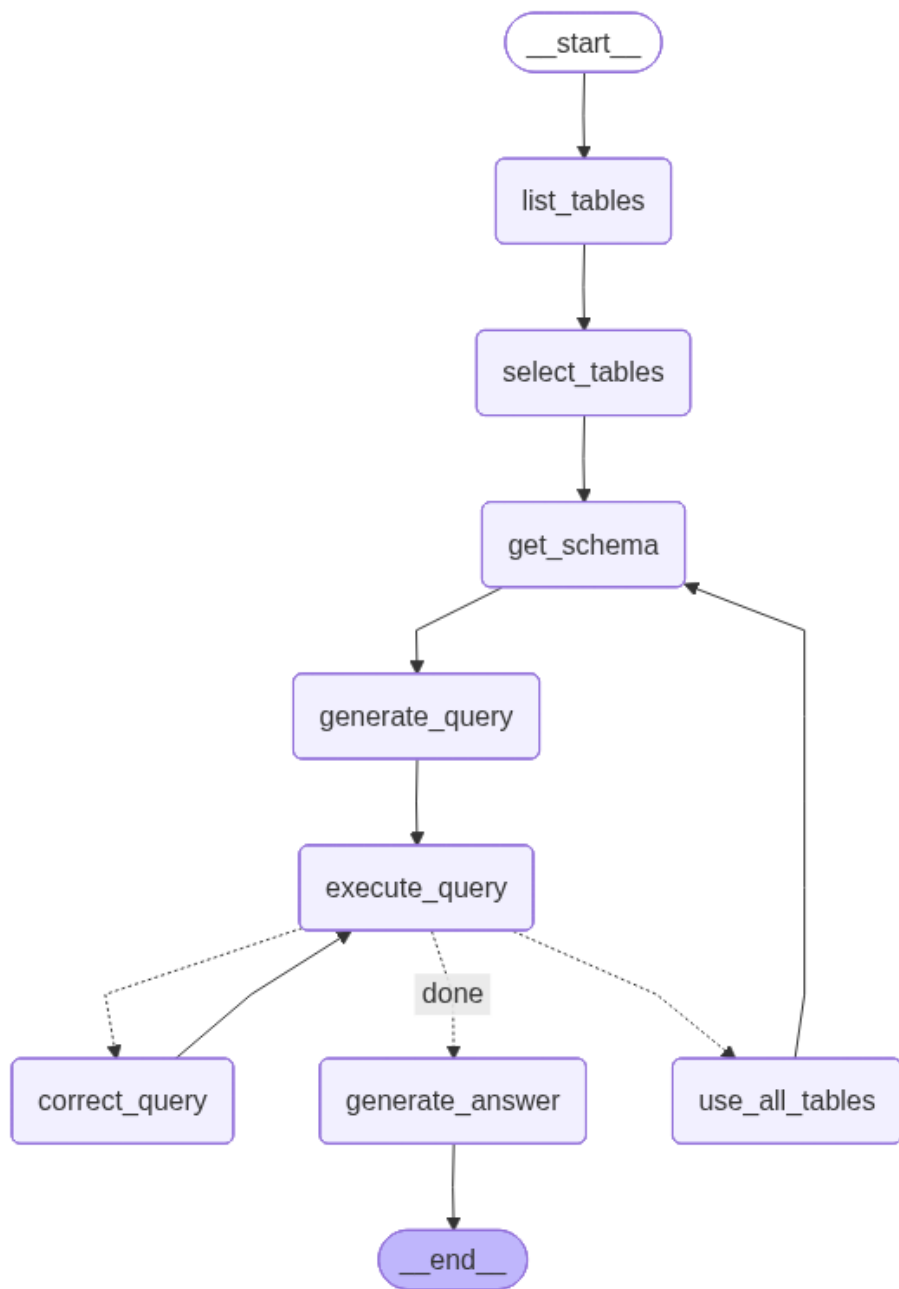
Zaimplementowany wariant Chain-of-Thought dodaje do generowania SQL krótki plan zapisany w bloku `<think>...</think>`. Używa wspólnego dla wszystkich strategii kroku schema linking (`list_tables` + `select_tables` z fallbackiem na pełny schemat), a właściwa modyfikacja CoT dotyczy tylko prompta w węźle `generate_query`. Plan ma zawierać maksymalnie kilka krótkich informacji:

- `tables` — istotne tabele,
- `joins` — potrzebne złączenia,
- `filters` — warunki filtrujące,
- `aggregation_ordering` — `GROUP BY` / `HAVING` / `ORDER BY` / `LIMIT`,
- `output` — zwracane kolumny lub wyrażenia agregujące.

Po planie model zwraca jedno zapytanie SQL `SELECT`. Implementacja wyodrębnia plan do osobnego pola diagnostycznego `reasoning_trace`, usuwa go z odpowiedzi modelu i zapisuje samo zapytanie SQL do ewaluacji. Dzięki temu można analizować sposób planowania modelu, ale format predykcji pozostaje zgodny z wymaganiami benchmarku.

Dla `qwen/qwen3-32b` zastosowano wariant SQL-only — model jest proszony o myślenie wewnętrzne bez wypisywania reasoningu, a pole `reasoning_trace` pozostaje wtedy puste.

W razie błędu wykonania lub odrzucenia przez walidator bezpieczeństwa, agent uruchamia wspólny węzeł `correct_query` (domyślnie do dwóch prób). Korektor



Rysunek 1: Graf agenta Chain-of-Thought

otrzymuje pytanie, schemat, poprzedni SQL i komunikat błędu — plan z `<think>` nie jest mu udostępniany.

Wariant CoT uruchamiamy flagą `--reasoning-mode cot`.

Wariant Plan and Solve

Wariant Plan and Solve rozdziela proces generowania zapytania SQL na dwa etapy: planowanie i rozwiązanie.

Krok planowania zawiera w prompcie informacje o pytaniu użytkownika, nazwach i schematach istotnych tabel, wynikiem jest tekstowy plan rozwiązania.

Krok rozwiązanie ma za zadanie wyprodukowanie zapytania SQL, korzystając z informacji o pytaniu użytkownika, nazwach i schematach istotnych tabel, a także z wygenerowanego wcześniej planu.

W przypadku błędu, poprawa zapytania również jest rozbita na dwa etapy. Prompt jest dodatkowo wzbogacony o komunikat błędu, plan i zapytanie SQL, które spowodowały błąd. Model ma za zadanie wygenerować poprawiony plan, a następnie poprawione zapytanie SQL.

Wariant ReAct-lite

ReAct-lite jest uproszczonym wariantem ReAct, w którym jedyną akcją modelu jest wygenerowanie zapytania SQLite `SELECT`. Pojedyncza iteracja składa się z trzech elementów:

- rozumowania nad pytaniem użytkownika, schematem i historią poprzednich prób,
- akcji, czyli wygenerowania jednego zapytania SQL,
- obserwacji, czyli wyniku wykonania zapytania albo komunikatu błędu.

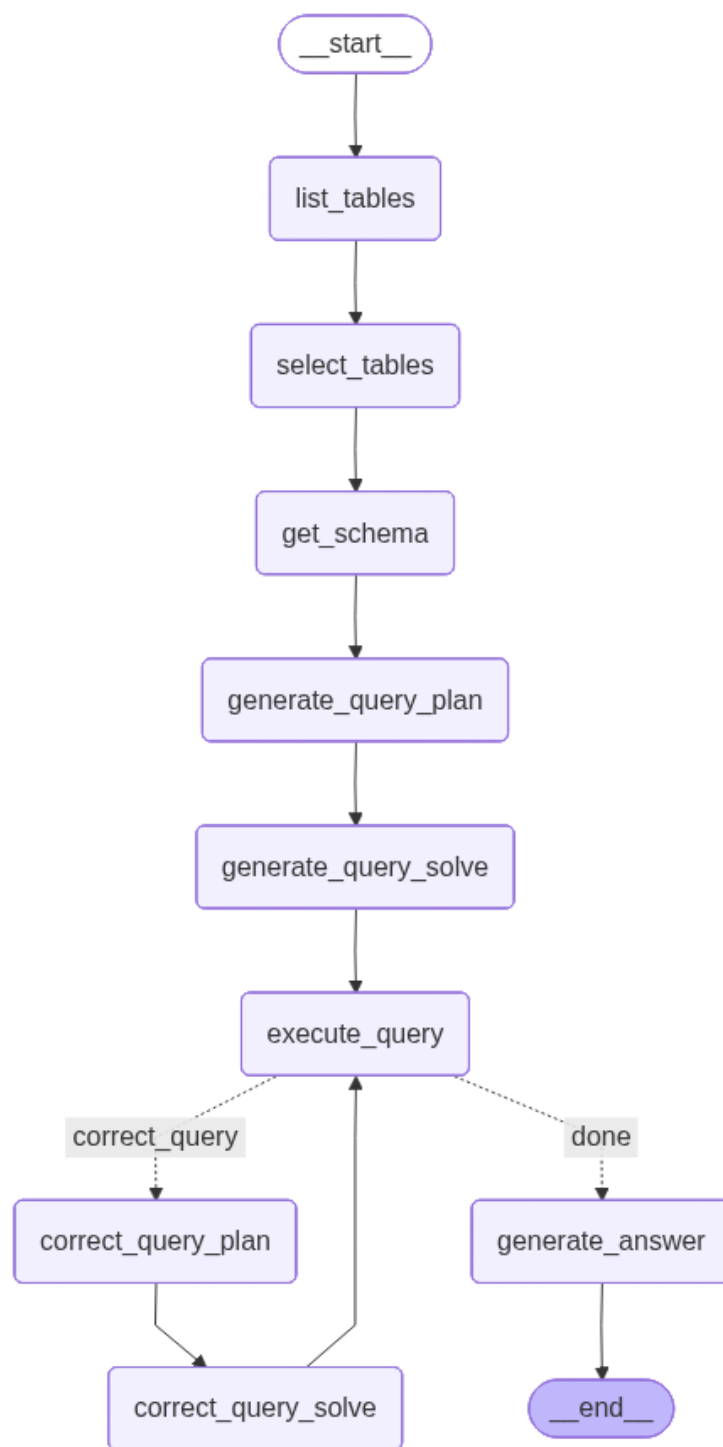
Jeżeli zapytanie zakończy się błędem, agent zapisuje poprzednie *Thought/Action/Observation* w historii i generuje kolejną próbę. W przeciwieństwie do CoT korekta korzysta więc nie tylko z błędnego SQL, ale również z wcześniejszego rozumowania i obserwacji. W przeciwieństwie do Plan-and-Solve nie powstaje oddzielny, stały plan przed wykonaniem zapytania.

Dla modeli lokalnych dopuszczono format `Thought: ...` oraz `SQL: ....`. Dla `groq-qwen3-32b` użyto wariantu SQL-only, ponieważ widoczne uzasadnienia mogły zakłócać ewaluację Spidera.

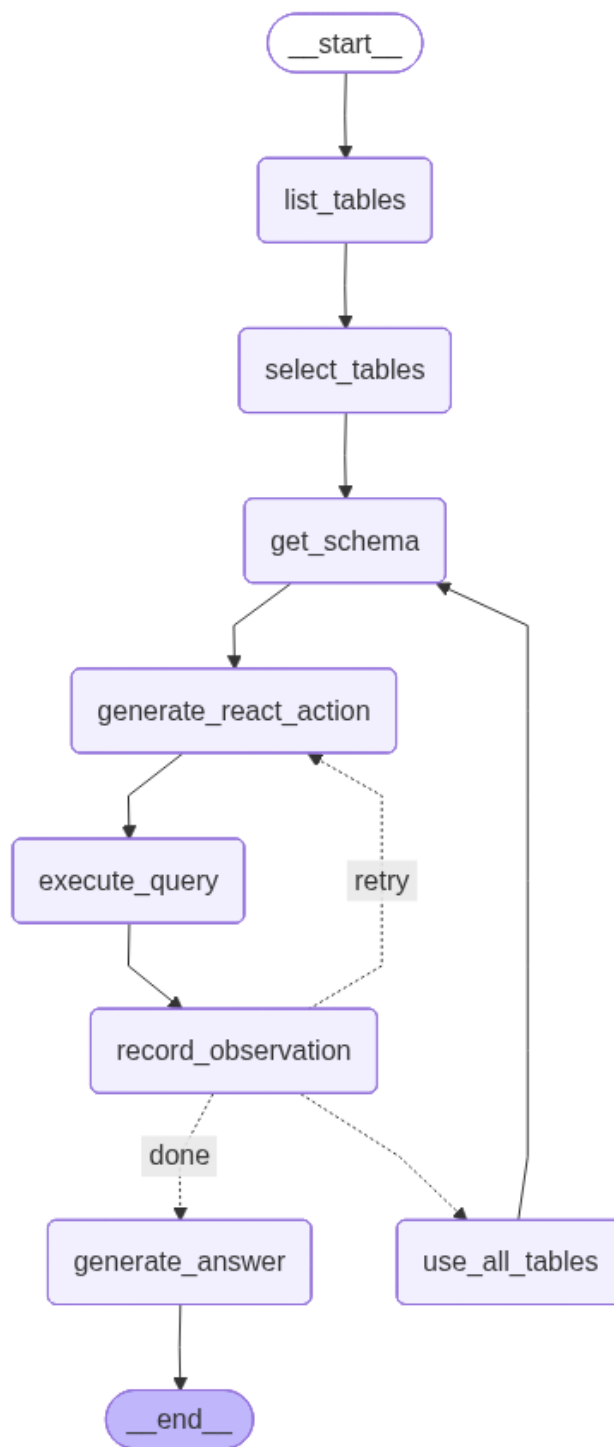
Algorytm self-correction

Self-correction jest uruchamiany, gdy zapytanie nie wykona się poprawnie albo zostanie odrzucone przez walidator bezpieczeństwa. Dla błędów typu `no such column` lub `no such table` agent może najpierw przełączyć się na pełny schemat bazy, ponieważ pierwotnie wybrane tabele mogły być niewystarczające.

Następnie model otrzymuje:



Rysunek 2: Graf agenta Plan and Solve



Rysunek 3: Graf agenta ReAct-lite

- pytanie użytkownika,
- schemat bazy danych,
- poprzednie zapytanie SQL,
- komunikat błędu.

Na tej podstawie generuje poprawione zapytanie. Domyślnie liczba prób korekty jest ograniczona, na przykład do dwóch. Pozwala to porównać wariant bez korekty, wariant z jedną próbą oraz wariant z większą liczbą prób.

Postprocessing zapytań SQL

Przed zapisaniem predykcji odpowiedź modelu jest czyszczona, aby do ewaluacji trafiło wyłącznie zapytanie SQL. Jest to potrzebne, ponieważ modele potrafią zwracać komentarze, markdown, fragmenty rozumowania albo znaczniki typu `<think>`.

Zaimplementowany postprocessing:

- usuwa bloki rozumowania, markdown i tekst przed właściwym `SELECT`,
- normalizuje aliasy tabel do formy z jawnym `AS`,
- usuwa backticki i aliasy kolumn wynikowych,
- odcina tekst po końcowym zapytaniu.

Dzięki temu parser Spidera ocenia wygenerowany SQL, a nie dodatkowy tekst znajdujący się w odpowiedzi modelu.

Struktura kodu źródłowego

- **agent** - pakiet z implementacją agenta
 - **common** - komponenty wspólne dla wszystkich wariantów agenta
 - * **agent** - abstrakcyjna klasa bazowa agenta
 - * **llm** - interfejs i konkretne adaptery do modeli językowych
 - * **logging_config** - konfiguracja logowania
 - * **modes** - definicja wariantów agenta
 - * **nodes** - węzły grafu agenta
 - * **state** - bazowy interfejs dla stanu agenta
 - * **utils** - funkcje pomocnicze
 - **chain_of_thought** - implementacja wariantu Chain-of-Thought
 - * **agent** - implementacja agenta
 - * **nodes** - implementacja węzłów specyficznych dla tego wariantu
 - * **state** - stan agenta specyficzny dla tego wariantu
 - **plan_and_solve** - implementacja wariantu Plan and Solve
 - * **agent, nodes, state**
 - **react_lite** - implementacja wariantu ReAct-lite
 - * **agent, nodes, state**
 - **agent** - budowanie agenta, skrypt demonstracyjny
 - **make_predictions** - skrypt do generowania predykcji dla zbioru Spider

- `eval` - moduł do ewaluacji wygenerowanych predykcji
 - kod zapożyczony z repozytorium <https://github.com/taoyds/spider>

Instrukcja obsługi

- Do zarządzania projektem Python, zależnościami: `uv`
- Do wykonywania komend: `just`
- Wszystkie komendy opisane w `Justfile`
 - lista wszystkich dostępnych `just -l`
 - opis skryptów `just <komenda> --help`
 - komendy uruchamiane np. `just test`
- Instalacja zależności: `just install`
- Pobranie zbiorów danych `just datasets`
- Demonstracyjne uruchomienie agenta, z odpowiedzią w języku naturalnym: `just agent`
- Przeprowadzenie ewaluacji
 - `just make_predictions` - generuje za pomocą agenta zapytania SQL dla zapytań w języku naturalnym i baz danych ze zbioru Spider.
 - `just eval` - oblicza miary jakości porównując wygenerowane i wzorcowe zapytania SQL

Testy

Przykłady ze zbioru testowego

- *How many items are shipped?*
 - `SELECT count(*) FROM Shipment_Items`
- *Find the details of the teachers who have taught the student with the earliest transcript issuance.*
 - `SELECT T1.teacher_details FROM Teachers AS T1 JOIN Classes AS T2 ON T1.teacher_id = T2.teacher_id JOIN Transcripts AS T3 ON T2.student_id = T3.student_id ORDER BY T3.date_of_transcript ASC LIMIT 1`
- *What are the ids of the documents that have between 2 and 4 related documents and how many related items are there?*
 - `SELECT Document_Object_ID , count(*) FROM Document_Subset_Members GROUP BY Document_Object_ID HAVING count(*) BETWEEN 2 AND 4;`
- *What are the receipt numbers for instances where both cakes and cookies were purchased?*
 - `SELECT T1.receipt FROM items AS T1 JOIN goods AS T2 ON T1.item = T2.id WHERE T2.food = \"Cake\" INTERSECT SELECT T1.receipt FROM items AS T1 JOIN goods AS T2 ON T1.item = T2.id WHERE T2.food = \"Cookie\"`

Wyniki benchmarku spider

Tabele przedstawiają wyniki dla partycji testowej zbioru benchmarkowego Spider.

Execution accuracy

Strategia	Model	Wszystkie	Łatwe	Średnie	Trudne	Bardzo trudne
PaS	Qwen	0.364	0.611	0.383	0.220	0.182
PaS	Llama	0.075	0.168	0.054	0.052	0.034
CoT	Qwen	0.406	0.636	0.421	0.281	0.227
CoT	Llama	0.068	0.068	0.049	0.043	0.031
ReAct	Qwen	0.428	0.606	0.474	0.300	0.246

Exact match accuracy

Strategia	Model	Wszystkie	Łatwe	Średnie	Trudne	Bardzo trudne
PaS	Qwen	0.103	0.219	0.112	0.043	0.008
PaS	Llama	0.019	0.081	0.002	0.002	0.000
CoT	Qwen	0.146	0.304	0.152	0.067	0.025
CoT	Llama	0.017	0.060	0.007	0.006	0.000
ReAct	Qwen	0.238	0.449	0.270	0.104	0.059

Wyniki z porównania 40 zapytań na zbiorze testowym

Execution accuracy

Strategy	Model	All	Easy	Medium	Hard	Extra hard
CoT	Qwen	0.250	0.400	0.200	0.400	0.000
ReAct-lite	Qwen	0.250	0.600	0.200	0.200	0.000
Plan-and-Solve	Qwen	0.200	0.300	0.200	0.200	0.100
CoT	Groq	0.550	0.800	0.500	0.600	0.300
	Qwen3-32B					
ReAct-lite	Groq	0.600	0.800	0.700	0.500	0.400
	Qwen3-32B					
Plan-and-Solve	Groq	0.475	0.800	0.400	0.500	0.200
	Qwen3-32B					

Exact match accuracy

Strategy	Model	All	Easy	Medium	Hard	Extra hard
CoT	Qwen	0.125	0.300	0.100	0.100	0.000
ReAct-lite	Qwen	0.175	0.400	0.100	0.200	0.000
Plan-and-Solve	Qwen	0.075	0.200	0.000	0.100	0.000
CoT	Groq	0.475	0.800	0.400	0.600	0.100
	Qwen3-32B					
ReAct-lite	Groq	0.475	0.800	0.500	0.500	0.100
	Qwen3-32B					
Plan-and-Solve	Groq	0.275	0.600	0.200	0.300	0.000
	Qwen3-32B					

Partial matching F1

Strategy	Model	Select	Where	Group	Order	Keywords
CoT	Qwen	0.381	0.438	0.276	0.211	0.414
ReAct-lite	Qwen	0.469	0.462	0.343	0.435	0.576
Plan-and-Solve	Qwen	0.349	0.370	0.400	0.381	0.483
CoT	Groq	0.838	0.812	0.667	0.696	0.765
	Qwen3-32B					
ReAct-lite	Groq	0.789	0.765	0.650	0.750	0.817
	Qwen3-32B					
Plan-and-Solve	Groq	0.743	0.667	0.732	0.800	0.738
	Qwen3-32B					

Porównanie wyników z literaturą

Na stronie <https://yale-lily.github.io/spider> przedstawiony jest ranking wyników na zbiorze testowym Spider.

Execution accuracy Najwyższy wynik na poziomie 91.2% osiągnął model MiniSeek, natomiast nie jest dostępny artykuł ani kod źródłowy. Najwyższy wynik z dostępną publikacją i kodem źródłowym osiągnął model DAIL-SQL + GPT-4 + Self-Consistency: 86.6%

Exact match Najwyższy wynik na poziomie 81.5% osiągnął model MiniSeek. Najwyższy wynik z dostępną publikacją i kodem źródłowym osiągnął model Graphix-3B + PICARD (DB content used): 74.0%

Wnioski

1. Z trzech zmiennych (model, strategia, korekta) wybór modelu ma na pierwszy rzut oka największy wpływ na wyniki. Na przeprowadzonym teście `diagnostic_test_40` model `qwen/qwen3-3b` osiągał wyniki ~2x większe niż odpowiednik danej strategii na modelu `Qwen 1.5B`.
2. W przypadku modeli i testów lokalnych widać, że strategia ReAct osiąga najlepsze wyniki, za którymi podąża CoT, a za nim Plan and Solve. Sam ten fakt również może wynikać z podejścia do implementacji self-correction, gdzie ReAct najlepiej (w pełni) go wykorzystuje.
3. Llama 1B, niezależnie od przyjętej strategii osiąga gorsze wyniki niż bazowy model `Qwen 1.5B`.
4. Trudność zadań istotnie wpływa na zdolność małych modeli do stworzenia odpowiednich zapytań. Być może bardziej złożone zapytania SQL (INTERSECT / EXCEPT / UNION itp.) są zbyt trudne dla małych modeli językowych.
5. Zyski z użytku strategii uwydatniają się, gdy model bazowy ma jakąkolwiek zdolność do generowania kwerend SQL (vide case Llamy).

Bibliografia

- [1] T. Yu *i in.*, „Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. 2018. Dostępne na: <https://arxiv.org/abs/1809.08887>
- [2] J. Tian *i in.*, „Toward Real-World Table Agents: Capabilities, Workflows, and Design Principles for LLM-based Table Intelligence”. 2025. Dostępne na: <https://arxiv.org/pdf/2507.10281>
- [3] A. Su *i in.*, „TableGPT2: A Large Multimodal Model with Tabular Data Integration”. 2024. Dostępne na: <https://arxiv.org/abs/2411.02059>
- [4] V. Zhong, C. Xiong, i R. Socher, „Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. 2017. Dostępne na: <https://arxiv.org/abs/1709.00103>
- [5] J. Li, B. Hui, *i in.*, „Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs”. 2023. Dostępne na: <https://arxiv.org/abs/2305.03111>
- [6] Y. Shi i et al. Tang, „A Survey on Employing Large Language Models for Text-to-SQL Tasks”. 2024. Dostępne na: <https://arxiv.org/abs/2407.15186>
- [7] M. Pourreza i D. Rafiei, „DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction”. 2023. Dostępne na: <https://arxiv.org/abs/2304.11015>

- [8] L. Wang *i in.*, „Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. 2023. Dostępne na: <https://arxiv.org/abs/2305.04091>
- [9] S. Yao, J. Zhao, *i in.*, „ReAct: Synergizing Reasoning and Acting in Language Models”. 2022. Dostępne na: <https://arxiv.org/abs/2210.03629>
- [10] LangChain, „Build a SQL agent”. [Online]. Dostępne na: <https://docs.langchain.com/oss/python/langchain/sql-agent#run-your-agent-in-studio>
- [11] LangChain, „Build a custom SQL agent”. [Online]. Dostępne na: <https://docs.langchain.com/oss/python/langgraph/sql-agent>
- [12] H. Touvron *i in.*, „Llama 3: Open Foundation and Instruction Models”. 2024. Dostępne na: <https://arxiv.org/abs/2302.13971>
- [13] K. Ociepa, Ł. Flis, R. Kinas, K. Wróbel, i A. Gwoździej, „Bielik v3 small: Technical report”, *arXiv preprint arXiv:2505.02550*, 2025.